

Week 1 - Day 5b: Challenges

This notebook contains some more challenging problems.

In each of the exercises, your task is to write a function that solves the problem. The function `check_encrypted` will print a readable message if you got the right answer, and otherwise it will print some strange characters. There are two test cases in each problem, and solving the second one can be much harder than solving the first.

First, run the code below. Do not modify the code, it is only used for testing.

```
# This function is for testing, do not modify
def check_encrypted(answer, secret):
    answer = str(answer)
    result = []
    offset = sum([ord(c) for c in answer])
    for i in range(len(secret)):
        result.append(chr(((offset + ord(answer[i%len(answer)])) +
ord(secret[i]) - 64)%224 + 32))
    print("".join(result))
```

Exercise 1: Repeating number []

You are given two positive integers n and m . What is the minimum number of times you need to concatenate n with itself so that the result is a multiple of m ?

Example: if $n=10$ and $m=3$, then the answer is 3 because 101010 is a multiple of 3, whereas 10 and 1010 are not.

```
# TODO: Your code here
def repeating_number(n, m):
    pass

# Use this code cell to test your solution. Do not modify.
n1 = 2024
m1 = 139
check_encrypted(repeating_number(n1, m1), "→xÔÚÎËÝ«ÊÖÛÎÚ(")

n2 = 7345711
m2 = 123456789
check_encrypted(repeating_number(n2, m2), "g}q)0tu(3v|(5w|)>ur(6")
```

Exercise 2: Laptops []

In the lab there are n laptops in a long row. Each laptop can be either up or down. Daniel wants the lab to look more orderly, so he asks the TAs to make it so that the row of laptops consists of one row of laptops that are down followed by one row laptops that are up.

You are given a string of ones and zeros. Your task is to calculate the minimum number of characters that need to be changed so that the string is on the form `000...1111`. The string has to start with a `0` and end with a `1`.

Example: if the string is `0100110`, then the answer is 2. We can change the second and last character to get the row `0000111`. Now, we have a row of zeros followed by a row of ones, which is what we want. Therefore, the answer is 2.

```
# TODO: Your code here
def laptops(row):
    pass

# Use this code cell to test your solution. Do not modify.
import random
random.seed(2024)

row1 = "".join(["01"[random.randint(0,1)] for _ in range(1000)])
check_encrypted(laptops(row1), "(ç  []Pv©[]")

row2 = "".join(["01"[random.randint(0,1)] for _ in range(2*10**5)])
check_encrypted(laptops(row2), "àô%ý$³æ$%'û û")
```

Exercise 3: Super Root []

Write a function `find_root(m)` that takes a **positive integer** m and finds a **positive integer** n such that $\sqrt{n}+n^2=m$. If no such number n exists, the function should return -1 instead.

Example: `find_root(84)` is 9, because $\sqrt{9}+9^2=3+81=84$.

```
# TODO: Your code here
def find_root(m):
    pass

# Use this code cell to test your solution. Do not modify.
check_encrypted(find_root(15240969373582152), "=JP0âGSSKçå8RâVSQ\
KâXKGãPFXMJVãVHWYIGXÎiâ\SZæSNKKVãRJKJâXRâVTJKJâYSâ\
SZXæHSGGã[NZNâFLPDV^æYJEUEK")
check_encrypted(find_root(9999999996000000000599999999970000000000),
"p|}p}Mn|}{ (")
```